

Fast hardware architecture for 2D separable convolution operations

Debasish Mukherjee, *Student Member, IEEE* and Susanta Mukhopadhyay

Abstract—Two-dimensional (2D) convolution is a widely used low-level image processing operation especially in spatial filtering, sharpening and edge detection. Real-time implementation of convolution operation is a challenging task due to its increasing computational cost with increasing kernel and input image size. This brief focuses on the real-time implementation of separable convolution operation based on distributed arithmetic with support for processing high-resolution images using large size kernel. The architecture processes pixels immediately whenever available from external memory achieving low processing time compared to existing works. Two FPGA architectures namely Single Window Separable Convolution (SWSC) architecture and Multi-Window Separable Convolution (MWSC) architecture are proposed. These architectures are realized based on concurrent access of stored pixels and parallel processing of overlapped window in order to attain higher performance/area utilization. Implementation results show that SWSC architecture achieves better processing time and area utilization at the cost of lower throughput compared to MWSC architecture.

Index Terms—Separable Convolution, Distributed Arithmetic, FPGA.

I. INTRODUCTION

TWO-DIMENSIONAL (2D) convolution is a fundamental operation in image and video processing applications [1]. Convolution operation involves two operands: input image of size $M \times N$ and a convolution kernel of size $R \times S$. The operation involves a sum of the products of each pixel and kernel weights within the convolution window with the window being moved lexicographically over the entirety of the image. Although conceptually simple, the operation requires $R \times S$ number of multiplications and $(R \times S) - 1$ number of additions with $R \times S$ access to the input image. Clearly, the number of operations increases with increase in mask size. Thus, for real-time consideration, novel design strategies must be devised in order to enhance the performance of 2D convolution operations.

To address the above issues, several high performance and area efficient architectures have been devised in the literature [2]–[6]. These architectures can be broadly categorized into (i) Systolic array [2] and (ii) Multiplier-less implementations such as ROM based [3] and Shift-accumulate logic [4]. Studies show that multiplier-less implementations provide better area utilization as compared to embedded DSP multipliers [3]. Separability of convolution kernel is a popular topic as most of the 2D kernel such as Sobel kernel, Gaussian kernel, Laplacian kernel etc. are strictly separable. A 2D convolution kernel is separable if it can be decomposed into two 1D kernels. The computational complexity is reduced by processing two 1D kernels instead of one 2D kernel. Although much of the work

has been carried out in processing arbitrary kernel, hardware optimization of separable kernel has not been exhaustively covered in the literature. Another alternative strategy for optimizing area/performance utilization is obtained via buffering scheme that stores pixels prior to processing. Buffering scheme combined with concurrent access of overlapped convolution kernel, Multi Window Partial Buffering (MWPB) scheme [5] achieves a good balance between the available on-chip resources and external memory bus bandwidth. Reducing the number of convolution operations by using a smaller mask instead of one large mask, combined with existing buffering scheme provide better area/performance trade-off compared to state-of-the-art methods [6].

All the previous optimization techniques either scan the input pixel in arbitrary order (shift and accumulate and ROM-Based) or delays processing of pixels by either storing them into the input buffer (buffering scheme) or delay unit (systolic array) prior to processing. This in effect increases the processing time and reduces the throughput. The main contributions of the paper are summarized below:

- 1) A parameterized Single Window Separable Convolution (SWSC) architecture optimized for area and performance utilization.
- 2) SWSC architecture is designed using 1D horizontal unit, 1D Vertical unit and an intermediate buffering for simultaneous access of stored pixels.
- 3) Based on Distributed Arithmetic (DA), the architecture for 1D horizontal and vertical kernel has been designed for processing large kernel size.
- 4) Multi-Window Separable Convolution (MWSC) architecture has been designed exploiting the overlapped region of convolution kernel resulting in high throughput compared to SWSC architecture.

In Section II, a DA based architecture for 2D convolution is described. Section III explains the performance analysis followed by comparison with existing implementations. Finally Section IV concludes this brief.

II. PROPOSED METHOD

In this section, we will first introduce DA strategy followed by the proposed architecture for 2D convolution. Throughout the discussion, we will consider an input image of size $M \times N$ and kernel of size $R \times S$.

A. Distributed Arithmetic based 2D convolution

Conventionally, multiplication in the convolution process is carried out by built-in multipliers found in modern FPGAs.

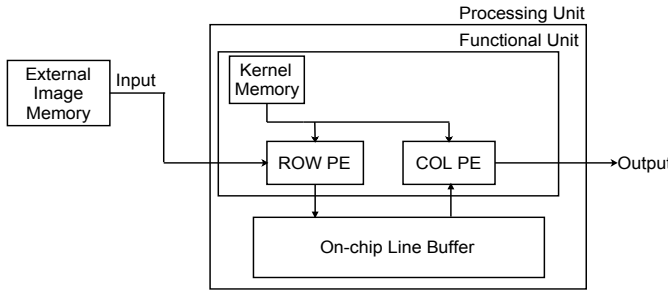


Fig. 1. Design overview of convolution architecture

The number of multiplier units increases exponentially with increase in kernel size. This fact can either limit the kernel size or force to use a high-cost FPGA device. A solution would be to provide a multiplier less approach that will implement large size kernel on the low-cost device. Distributed Arithmetic (DA) [7] which is a multiplier less approach that performs multiplication using precomputed look-up tables (LUTs). To understand the application of DA in convolution, let us consider three 1D vectors each with 8-bit representation $A = \{a_7 2^7 + a_6 2^6 + a_5 2^5 + a_4 2^4 + a_3 2^3 + a_2 2^2 + a_1 2^1 + a_0 2^0\}$, $B = \{b_7 2^7 + b_6 2^6 + b_5 2^5 + b_4 2^4 + b_3 2^3 + b_2 2^2 + b_1 2^1 + b_0 2^0\}$, $C = \{c_7 2^7 + c_6 2^6 + c_5 2^5 + c_4 2^4 + c_3 2^3 + c_2 2^2 + c_1 2^1 + c_0 2^0\}$ and three kernel weights w_1, w_2, w_3 . The results of convolution which is basically the multiplication of input vectors with the kernel weights can be expressed as:

$$Y = w_1(a_7 2^7 + a_6 2^6 + \dots + a_0 2^0) + w_2(b_7 2^7 + b_6 2^6 + \dots + b_0 2^0) + w_3(c_7 2^7 + c_6 2^6 + \dots + c_0 2^0)$$

$$Y = (w_1 a_7 + w_2 b_7 + w_3 c_7) 2^7 + (w_1 a_6 + w_2 b_6 + w_3 c_6) 2^6 + \dots + (w_1 a_0 + w_2 b_0 + w_3 c_0) 2^0 \quad (1)$$

The term in each parenthesis of equation 1 can be precomputed and stored in an LUT whose contents are generated for each bit of three input vectors. After shift and accumulate operation the product Y can be obtained. For 2D convolution, similar procedure is applied for row vector followed column vector. It may be noted that the final result can be obtained either in N cycles (serial addition) or in 1 cycle (parallel addition). In this article we have assumed parallel approach in order to increase throughput at the cost of nominal increase in area utilization. ROM size can act as a bottleneck and can practically limit the implementation to relatively small filters since ROM size grows exponentially as kernel size increases. For instance, for 16-bit input data with $M = 16$, $16 \times 2^{16} = 1$ Mbit of memory is required. To overcome the limitation, a single ROM unit is partitioned into several ROM units whose output are combined using a tree-based adder unit.

B. Design Considerations

1) *2D Convolution*: Fig. 1 shows the overview of the convolution architecture which consists of external image memory and processing unit. The input image MN is stored in the external memory unit. The processing unit is fully

parameterized and implemented on FPGA platform. It consists of several sub-modules such as functional unit and an on-chip line buffer. The functional unit is composed of kernel memory, ROW PE for processing of 1D kernel horizontally and COL PE for processing of 1D kernel vertically.

2) *Convolution Operations*: There are two types of operations: Horizontal 1D operation and Vertical 1D operation. The operations are implemented using DA in FPGA. Input pixels read from the external image memory are subjected to horizontal operation. Its results are stored in the on-chip line buffer. Finally, the vertical operation is performed to get the desired result.

Two principle operations are followed in order to enhance the performance of the separable convolution architecture.

- 1) Data stored in the on-chip buffer are concurrently accessed.
- 2) Parallelism in the processing of pixels belonging to the overlapped region of the kernel that gives rise to speed up equal to the number of overlapped kernels.

C. Proposed architecture for 8 bit $1 \times N$ Convolution

Fig. 2 shows the block diagram of 8-bit 1×7 convolution. For a more general description of M -bit $1 \times N$ kernel, N number of M -bit shift registers (D_0 to D_{N-1}) and $2M$ number of 16×8 ROM units along with a tree based pipelined adder unit is required. For simplicity, we will describe the block diagram of 8-bit 1×7 kernel. At the rising edge of the clock pulse, current input pixel and all the previous pixels are stored in the shift register (D_0 to D_6). As shown in the figure new data is shifted in through the D_6 register and shifted out through the D_0 register with the output of D_i connected to the input of D_{i-1} . A single 128×8 ROM is partitioned into two 16×8 ROMs whose outputs are combined using a 2-input adder module. To enhance the performance of the system, some measure of area optimization is employed by using Carry Save Adder (CSA) and Look-Ahead Carry Adder (LCA) module instead of adder module for combining the results from more than two ROM units. Each bit from D_i is grouped into k -data clusters to address ROM memories through the decoder unit. For instance, the first ROM is addressed by bits D_3 to D_6 and the second ROM by D_0 to D_2 . The two ROM units are addressed at the same time using a single clock. Thus all the data are generated from ROM unit in parallel and in one clock cycle. To complete the operation for 1×7 convolution, results are shifted and added through CSA and LCA modules. CSA and LCA modules are used to achieve low FPGA area utilization compared to the conventional adder module. Furthermore, the modules are pipelined in order to increase maximum clock frequency and throughput by reducing the critical path length.

D. Single Window Separable Convolution Architecture

Single Window Separable Convolution (SWSC) Architecture is shown in Fig. 3. It consists of on-chip line buffer, row-wise processing element (ROW PE), column-wise processing element (COL PE), De-multiplexer module (DEMUX), line Counter Module and MERGE UNIT. ROW PE and COL PE

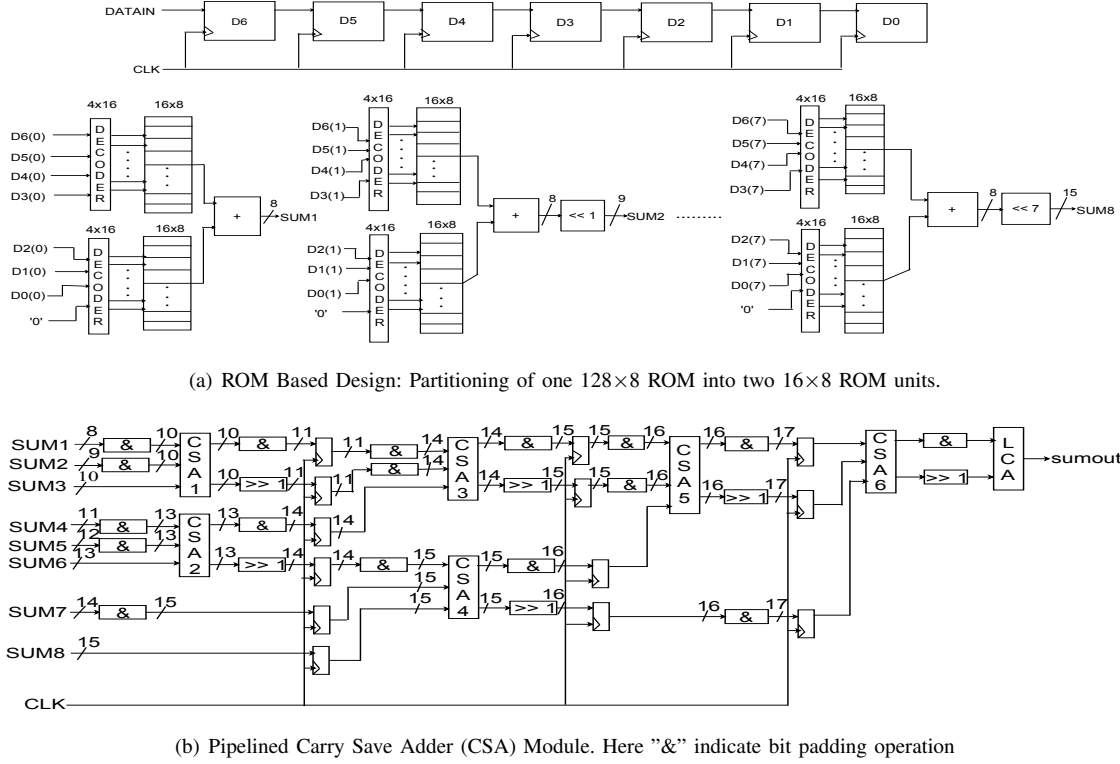


Fig. 2. 1×7 convolution module

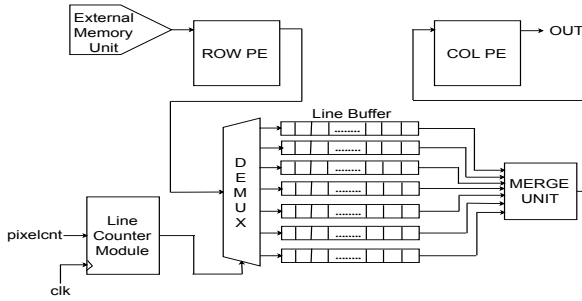


Fig. 3. Block diagram of Single Window Separable Convolution (SWSC) Architecture

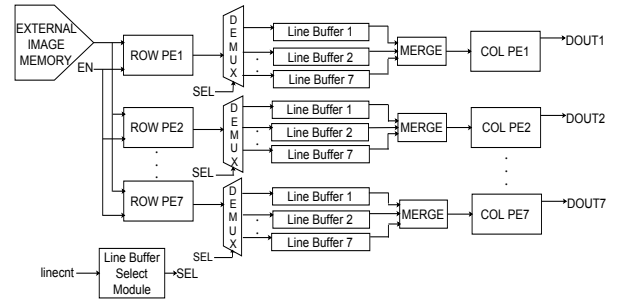


Fig. 4. Block diagram of Multi Window Separable Convolution Architecture

are implemented using DA based approach (Fig. 2). Input pixels read from external memory unit are passed to ROW PE which stores the results of processing $1 \times S$ kernel onto on-chip line buffer. The partial results from on-chip buffers are concurrently accessed which is later combined through MERGE UNIT to COL PE. For a given $R \times S$ kernel, at most R line buffers are required. At each clock cycle, results from ROW PE are stored on to a particular buffer which is determined by selection signal in DEMUX unit. Selection signal is generated based on Line Counter Module which is modulo R -counter where R is the row dimension of kernel incremented for each image line read.

E. Multi Window Separable Convolution Architecture

The block diagram of Multi-Window Separable Convolution (MWSC) architecture is shown in Fig. 4. The basic idea behind

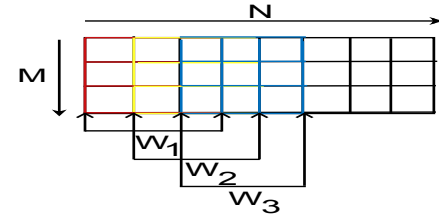


Fig. 5. Overlapping of windows for three consecutive operations. Here w_i refers to the i^{th} window

the concurrent processing of several kernels is that there is an overlapped region of pixels formed when two or more consecutive windows are processed. Fig. 5 shows overlapping of windows during three consecutive kernel operations. The pixel belonging to each of these windows can be broadcast for simultaneous processing. Thus, at most R consecutive

windows are processed resulting in a throughput of $f \times R$ where f is the operating frequency. Fig. 4 shows the architecture for the 7×7 kernel. Input from external memory is broadcast to the horizontal processing unit (HPU) which is enabled by EN signal. Once enabled each HPU output results on to line buffer. The intermediate results stored in line buffers are passed to vertical processing unit (VPU) through MERGE UNIT. The Output from VPU gives the results of processing R consecutive kernels along the column direction.

III. RESULTS AND DISCUSSION

The proposed 2D convolution has been prototyped on virtex 6 (xc6vlx240t-2ff784) FPGA and all the hardware modules are synthesized using Xilinx 14.2 ISE. The Design is modeled using VHDL and simulated using Xilinx simulator ISIM.

TABLE I
AREA AND PERFORMANCE UTILIZATION OF SINGLE WINDOW SEPARABLE CONVOLUTION ARCHITECTURE FOR 256×256 IMAGE SIZE, VARIED KERNEL SIZE

Kernel	3×3	7×7	11×11	21×21
Slice Flip Flop	230/301440 (1%)	390/301440 (1%)	884/301440 (1%)	2444/301440 (1%)
Slice LUTs	177/150720 (1%)	621/150720 (1%)	1205/150720 (1%)	4264/150720 (2%)
RAM BUFFER	3/832 (1%)	7/832 (1%)	11/832 (1%)	21/832 (2%)
Power (W)	0.159/3.041	0.225/3.109	0.288/3.175	0.539/3.437
Latency (clock)	532	1594	2681	5836
Throughput (Mpx/s)	18.2	12.5	8.75	7.26
Processing Time (msec)	0.27	0.34	0.38	0.47

Table I and II presents Area and Performance utilization for 256×256 image and varied kernel size for SWSC and MWSC architecture. Area utilization is measured in terms of (i) Slice Flip Flop (ii) Slice LUTs (iii) RAM BUFFER and (iv) Power while performance utilization is measured in terms of (i) Latency (ii) Throughput and (iii) Processing time. These results are obtained from post-place and route report after the implementation stage of the synthesizer tool. The efficacy of the design is analyzed by varying kernel size from 3×3 (small) to 21×21 (large). It can be pointed out that a gradual increase in the percentage of FPGA resource utilization occurs for processing small to large sized kernel. This is due to the fact that several hardware modules are concurrently re-utilized resulting in increasing percentage of area utilization. The measured percentage of area utilization is low (about 1% Slice FF, 2% Slice LUTs, 21×21 kernel for SWSC and 16% Slice FF, 52% Slice LUTs, 21×21 kernel for MWSC) which concludes the fact that larger kernel configurations can be supported on the given FPGA device. Table I and II shows dynamic power with respect to total power. Power consumption is measured using Xilinx Power Analyzer (XPA) tool. XPA uses NCD and VCD file for power

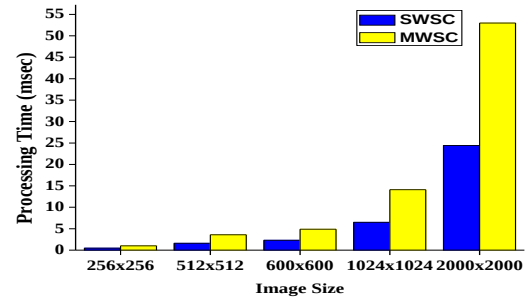


Fig. 6. Comparison of processing Time for SWSC and MWSC architecture, Kernel size: 21×21

analysis. NCD output file is generated after place and route (P&A) whereas VCD file is a simulation activity file which is generated after post-place and route simulation. Throughput is the average number of results produced per clock cycle. Thus, throughput measured in Mpx/sec is given by the following equation:

$$Throughput = \frac{F}{C} \times P \quad (2)$$

where F is the maximum clock frequency, C is the average clock cycle time to produce a result and P is the parallelism factor. Here P equals to one for SWSC architecture whereas it is equal to R for MWSC architecture. Latency is the average clock cycle time required to process one fixed size kernel. Latency depends on the size of the kernel which can be easily seen from table I and II. Processing time measured in millisecond (msec) is the time taken to process an image of given dimension. It is worthwhile to mention that the proposed architecture does not delays processing of pixels resulting in low processing time even for large size kernel.

TABLE II
AREA AND PERFORMANCE UTILIZATION OF MULTI-WINDOW SEPARABLE CONVOLUTION ARCHITECTURE FOR 256×256 IMAGE SIZE, VARIED KERNEL SIZE

Kernel	3×3	7×7	11×11	21×21
Slice FF	676/301440 (1%)	3401/301440 (1%)	11197/301440 (3%)	50471/301440 (16%)
Slice LUTs	501/150720 (1%)	5884/150720 (3%)	14323/150720 (9%)	78407/150720 (52%)
RAM BUFFER	9/832 (1%)	49/832 (6%)	121/832 (14%)	441/832 (53%)
Power (W)	0.423/3.316	0.864/3.776	1.666/4.613	1.255/4.184
Throughput (Mpx/s)	65.9	91.1	114.7	85.05
Latency (clock)	606	1585	2462	5561
Processing Time (msec)	0.27	0.38	0.4	0.99

Fig. 6 shows processing time of 21×21 kernel with respect to varying image size for SWSC and MWSC architecture. The architecture is able to process 41 and 19, 2000×2000 images per second for SWSC and MWSC architecture respectively. This concludes the fact that the architecture does

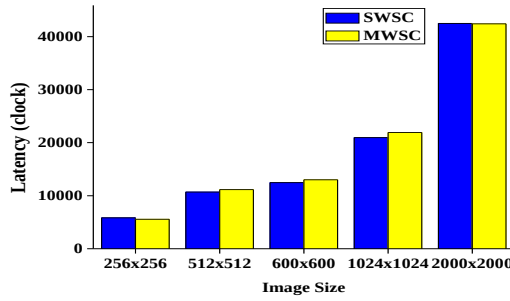


Fig. 7. Comparison of latency for SWSC and MWSC architecture, Kernel size: 21×21

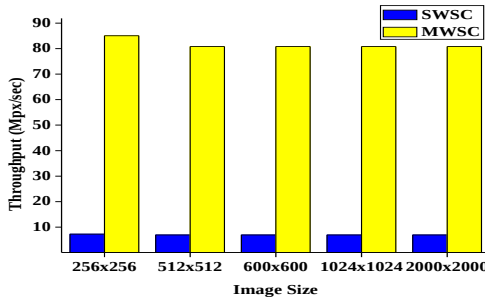


Fig. 8. Comparison of throughput for SWSC and MWSC architecture, Kernel size: 21×21

not deteriorate the performance for processing high resolution images with large kernel size. Fig. 7 and Fig. 8 show latency and throughput comparison for SWSC and MWSC architecture. Although latency remains almost same but simultaneous access of overlapped kernels achieves higher throughput in MWSC as compared to SWSC architecture.

Table III summarizes the performance results of the proposed architecture (SWSC and MWSC) and other related state-of-the-art works. It is worth to point out that for a fair comparison, the proposed architecture is implemented using similar image resolution, kernel size and device configurations. Toledo architecture [3] is optimized to process large kernel size but an arbitrary scan of input image increases the processing time for high resolution images. Torres architecture [2] is based on systolic array that delays processing of pixels for synchronization of input pixel with kernel weights thereby increasing the processing time. Gonzalez et al. [8] and Wong et al. [9] achieves performance close to the proposed work but are optimized for processing small kernel size and any further increase in configurations will deteriorate the performance of the architecture.

IV. CONCLUSION

This paper presents a DA based architecture for 2D convolution. It is able to process 8-bit kernel coefficients of size up to 21×21 . The proposed architecture following sequential data access, does not delay processing of input pixels, thereby achieving low processing time compared to existing works. Both SWSC and MWSC architecture have been tested for kernel size up to 21×21 and it is possible to process larger kernels given the lower percentage area utilization of such high

TABLE III
COMPARATIVE ANALYSIS BETWEEN THE STATE-OF-THE-ART HARDWARE ARCHITECTURES

Method	Device Name	Image Size	Kernel Size	Processing time (msec)
Toledo et al. [3]	XCV4LX160-10	1920×1080	7×7	16.6
		1920×1080	11×11	16.6
Torres et al. [2]	XCV2000E-6	512×512	7×7	8.35
Gonzalez et al. [8]	Cyclone II	1200×720	7×7	9.7
Wong et al. [9]	XCV2V6000	640×480	3×3	1.5
		512×512	7×7	4.87
		640×480	3×3	1.8
		1920×1080	7×7	15.6
		1920×1080	11×11	15.6
		1200×720	7×7	9.5
SWSC	XC4VLX160-10	1920×1080	7×7	15.6
		1920×1080	11×11	15.6
MWSC	XC2V2000-4	1200×720	7×7	9.5
		1024×768	21×21	4.8
MWSC	XC6VLX-240t	1024×768	21×21	4.8
		1024×768	21×21	10.6

density device. Concurrent access of pixels belonging to the overlapped region of kernels provides better throughput for MWSC architecture but higher percentage area utilization reduces maximum clock frequency resulting in high processing time compared to SWSC architecture.

REFERENCES

- [1] Rafael C. Gonzalez and Richard E. Woods, "Digital Image Processing (2nd Ed)" Englewood Cliffs, NJ: Prentice Hall, 2002.
- [2] César Torres-Huitzil, and Miguel Arias-Estrada, "FPGA-based configurable systolic architecture for window-based image processing", EURASIP Journal on Applied Signal Processing vol. 2005, pp. 1024–1034, Jan. 2005.
- [3] F Javier Toledo-Moreo, and J Javier Martinez-Alvarez, and Javier Garrigos-Guerrero, and J Manuel Ferrandez-Vicente, "FPGA-based architecture for the real-time computation of 2-D convolution with large kernel size", Journal of Systems Architecture, vol. 58, no. 8, pp. 277–285, Jun. 2012.
- [4] Myung H Sunwoo, and Seong K Oh, "A multiplierless 2-D convolver chip for real-time image processing", The Journal of VLSI Signal Processing, vol. 38, no. 1, pp. 63–71, August 2004.
- [5] Hui Zhang, and Mingxin Xia, and Guangshu Hu, "A multiwindow partial buffering scheme for FPGA-based 2-D convolvers", IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 54, no. 2, pp. 200–204, February 2007.
- [6] Zhao-Bin Ma, and Yang Yang, and Yun-Xia Liu, and Anil Anthony Bharath, "Recurrently Decomposable 2-D Convolvers for FPGA-Based Digital Image Processing", IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 63, no. 10, pp. 979–983, October 2016.
- [7] Stanley A White, "Applications of distributed arithmetic to digital signal processing: A tutorial review", IEEE ASSP Magazine, vol. 6, no. 3, pp. 4–19, July 1989.
- [8] Abiel Aguilar-González, and Miguel Arias-Estrada, and Madaín Pérez-Patricio, and JL Camas-Anzueto, "An FPGA 2D-convolution unit based on the CAPH language", Journal of Real-Time Image Processing, pp: 1–15, October 2015, DOI: <https://doi.org/10.1007/s11554-015-0535-1>.
- [9] Sebastian C Wong, and Mark Jasunas, and David A Kearney, "Fast 2D convolution using reconfigurable computing", Int. Symp. on Signal Processing and its Applications, pp:791–794, 2005.

Debasish Mukherjee is currently a Ph.D research scholar in the department of Computer Science & Engineering, Indian Institute of Technology (Indian School of Mines), Dhanbad, India. He did his B.Sc with honours in Computer Science from University of Calcutta, M.Sc in Computer Science from West Bengal State University and M.Tech in Computer Science & Engineering from West Bengal University of Technology in 2008, 2010 and 2013 respectively. His research interests include hardware implementation of image processing algorithms.

Susanta Mukhopadhyay did his B.Sc. with Honours in Physics from Presidency College, Calcutta, B.Tech and M.Tech in Radiophysics and Electronics from the University of Calcutta and Ph.D. in Image Processing from the Indian Statistical Institute, Calcutta in 1988, 1992, 1994 and 2003 respectively. During 2001-2003 and 2004-2007 he worked at the Sanford Burnham Prebys Medical Discovery Institute, La Jolla, California, USA as postdoctoral researcher and Nanyang Technological University, Singapore, as research fellow respectively. His research area and interest include image and video processing, fMRI. He is currently working as Associate Professor in the Department of Computer Science and Engineering, Indian Institute of Technology (Indian School of Mines), Dhanbad, India.